The General

[22/04/26, 9:45:32 AM] ~Charlie: there needs to be a sick af boot up page where the particles swarm around the addictd logo and create the orb
[22/04/26, 9:45:47 AM] ~Charlie: but we'll take more on the call, but try and get a picture from that board and the sample footage
[22/04/26, 9:46:16 AM] ~Charlie: idea is the orb oscillates and reflects light (and that refracted light / "forecasted visions" are the features of the addictd tool and our socials etc
[22/04/26, 9:46:57 AM] ~Charlie: I think you get idea but just run with creativity too. Idea is that as you scroll the orb rotates in cool angles and then eventually hits a falling animation, shatters into the particles
[22/04/26, 9:47:08 AM] ~Charlie: and the format changes to like fog

These are the storyboard images and the text messages that were provided.
I want you to first go through the images and the messages and first report to me what you did understand from the context given. This is for the landing page of a company named addictd.ai and the co founder wants to trust me with this.
Act like a senior full stack developer and understand this project first.

Gemini's response:

This sounds like an incredibly high-impact, Awwwards-caliber landing page. Charlie is definitely looking for a "wow" factor, and as a senior developer, reading this immediately points me toward a heavy WebGL/Canvas-driven architecture.

Before I break down the technical translation of this vision, I do need to mention: **I can't currently see the storyboard images or the sample footage you referenced.** However, based purely on Charlie's text messages, I have a very clear picture of the user journey and the technical requirements.

Here is my report on how I understand the project, broken down into a technical user flow:

The User Journey & Technical Breakdown

1. The "Boot-Up" Sequence (Splash Screen)

- **The Vision:** A sick, dynamic entry where a swarm of particles pulls together to form the "addictd" logo, which then morphs into the core element: The Orb.
- **Dev Translation:** This requires a complex WebGL particle system. We'll likely need custom shaders (GLSL) to handle the physics of thousands of particles pulling together (flocking/attraction behaviors) without tanking the browser's framerate.

2. The Hero Section (The Oscillating Orb)

- **The Vision:** The newly formed orb sits in the center, oscillating and reflecting light. The refracted light rays act as "forecasted visions," visually projecting the features of the Addictd tool and your social links.
- **Dev Translation:** We are looking at a 3D sphere with a custom iridescent or glass/refractive material (likely using a library like Three.js). The "forecasted visions" could be handled by rendering HTML elements integrated into the 3D scene, or by mapping video/image textures onto planes that get distorted by the orb's light rays. It needs to feel alive and breathing.

3. The Scroll Experience (Interactive Storytelling)

- **The Vision:** As the user scrolls, the orb rotates at cool angles, reacting to the scroll depth rather than just playing a linear video.
- **Dev Translation:** Scroll-tied animations are essential here. We'd use an animation library like GSAP (GreenSock) linked to the ScrollTrigger plugin to scrub through the 3D camera angles and the orb's rotation based purely on where the user is on the page.

4. The Climax & Transition

- **The Vision:** Hitting a specific scroll point triggers a falling animation. The orb shatters back into particles, and the visual format transitions into a "fog."
- **Dev Translation:** This is the most technically demanding part. We need a shatter/fracture mechanic for the 3D model, reverting it back to the particle system from the boot-up sequence. The "fog" transition means blending out the sharp 3D geometry into volumetric fog shaders or CSS-based layered smoke effects, creating a smooth visual handoff to whatever content comes next.

The Stack I'd Recommend

To pull off this level of creativity and performance, a standard React/Next.js setup won't be enough on its own. We would need to build this using:

- **Three.js / React Three Fiber:** For rendering the 3D orb, the lighting, and the environment.
- **GLSL (Custom Shaders):** For the particle swarm, the glass/refractive light effects, and the fog transition.
- **GSAP (ScrollTrigger):** To tie the entire cinematic experience to the user's scroll wheel.

It is a fantastic concept, but it will require tight optimization to ensure it doesn't overheat users' laptops or crash on mobile browsers.

Since I don't have the visual references yet, what is the overall vibe and color palette you are aiming for with the orb and the fog (e.g., dark and cyberpunk, bright and ethereal, or sleek and minimalist)?

Got them! Seeing these visual references completely locks in the technical approach. The written concept was already ambitious, but these storyboards show that Charlie is going for a highly textured, ethereal, and almost chaotic sci-fi energy.

Here is my breakdown of the visual references and how we actually build this out from a frontend architecture perspective:

Visual Breakdown & Technical Mapping

1. The Purple Plasma Orb (Image 1)

- **The Visual:** This is highly volumetric. It’s not just a glass ball; it contains swirling, nebulous clouds and sharp, erratic lightning arcs.
- **The Tech:** To achieve this without loading a massive pre-rendered video, we have to write custom Fragment Shaders (GLSL) in Three.js. We'll use 3D Simplex noise to generate the swirling internal clouds and fractal noise for the lightning strikes. We can expose parameters (like speed, density, and color) to tweak the "aggressiveness" of the energy based on scroll velocity.

2. The Orbital Swarm & Grid (Image 2)

- **The Visual:** A central energy core surrounded by smaller glowing nodes, hovering over a highly structured grid with a realistic reflection.
- **The Tech:** This perfectly illustrates the "oscillating and reflecting light" phase Charlie mentioned. The smaller orbiting spheres are the ideal interactive UI elements to represent the "forecasted visions" (socials and tool features).
- **Design Integration:** Contrasting this highly dynamic, glowing 3D asset against a stark, high-contrast monochromatic layout will elevate the entire experience. Framing this swirling chaos within a strict, minimalist grid structure will give the landing page a massive, premium brutalist edge, making the orb feel like a powerful, contained energy source rather than just a floating graphic.

3. The Dense Nebula (Image 3)

- **The Visual:** A darker, moodier, and much denser cloud structure.
- **The Tech:** This looks exactly like the visual state right before, or during, the "fog transition" Charlie described at the end of the scroll. We can use GSAP ScrollTrigger to smoothly tween the shader uniforms (variables), shifting the bright plasma from Image 1 and 2 into this deep, murky nebula right before it shatters.

The Game Plan

This is a pure **React Three Fiber (R3F) + GSAP** project.

1. **Phase 1:** Build the base sphere geometry and write the core volumetric shader to get the swirling smoke and lightning looking right (matching the texture of Images 1 & 3).
2. **Phase 2:** Implement the orbiting particle system (Image 2) and tie their rotation and spacing to mathematical formulas (like a Fibonacci sphere) so they look organized but organic.
3. **Phase 3:** Layer HTML over the canvas for the typography and layout, tying the camera position and the orb's state to scroll events.

Since those orbiting nodes in the second image look perfect for housing the Addictd tool features and socials, should we build them as 3D objects that the camera zooms into, or project standard 2D HTML elements perfectly tracked over them so they are easier for users to read and click?

Ah, this makes a world of difference! Seeing the actual storyboards and those UI elements completely sharpens the technical approach. Charlie isn't just looking for a cool 3D background; this is a highly choreographed, interactive storytelling experience centered around their analytics product.

Here is my senior-level read on these new boards and how we need to pivot the architecture to pull this off:

1. The Boot-Up Catalyst (The "Storm")

- **What I'm seeing:** Images 2 and 3 lay out the exact intro sequence. The Addictd logo shatters into particles, whips into a high-speed tornado/mist with purple lightning, and then coalesces into the final, oscillating purple orb.
- **The Tech Execution:** This confirms we need a heavy reliance on custom GLSL shaders in React Three Fiber. Transitioning from discrete particles to a volumetric, misty orb with lightning zaps is tough to do with standard 3D meshes. We'll likely need to use a point cloud for the logo shatter, which gets caught in a noise field (the tornado), and then dynamically crossfade the material into a volumetric raymarching shader for the final misty orb.

2. The "Refraction" UI Cards (The Core Content)

- **What I'm seeing:** Images 4 and 5 show the actual product—sleek, dark-mode analytics cards (Retention Heat Map, viewership stats). Charlie wants these to appear as if they are refracted light or "bubbles" popping out of the orb as it rotates via scroll.
- **The Design & Tech Synergy:** To really make these data cards pop and feel legible, we should lean into a stark, high-contrast monochromatic UI layer sitting on top of the 3D canvas. Stripping away distracting background elements and housing these floating cards within a strict, minimalist grid structure will frame that chaotic, swirling purple orb perfectly. It anchors the chaos and gives the whole page a highly disciplined, premium

brutalist edge. For the execution, we can use `@react-three/drei`'s `<Html>` component to perfectly map standard DOM elements to 3D coordinates wrapping the orb, making them fully interactive while moving in 3D space.

3. The Background Dilemma

- **What I'm seeing:** Image 1 pitches a few environmental ideas: a stormy ocean, blowing tall grass, or deep space.
- **My Recommendation:** Given the intricate details of the purple storm orb and the dense data on the UI cards, putting a photorealistic ocean or blowing grass behind it might cause massive visual clutter (and tank performance). Sticking to a deep, stylized void or an extremely subtle, slow-moving starfield (space) will ensure the orb and the product features remain the absolute center of attention.

4. The "Sliding Door" Scroll Mechanics

- **What I'm seeing:** The notes mention a standard header that stays visible/hides on scroll, and the whole experience relies on a "sliding door effect" where scrolling forces the orb to oscillate back and forth, revealing different features.
- **The Tech Execution:** This is pure GSAP `ScrollTrigger` territory. We will pin the 3D canvas to the viewport and map the user's scroll progress (from 0 to 1) directly to the orb's rotation vector and the opacity/position of the HTML feature cards.

This is a beast of a project, but it will look incredible if we nail the pacing.

As the lead on this, what do you think our first sprint should be: tackling the complex GLSL shader for the storm-to-orb transition, or scaffolding out the Next.js/GSAP scroll timeline with a placeholder sphere just to get the "sliding door" layout locked in?

The Draft 01 Storyboard:

Here is the finalized, high-fidelity storyboard for the **addictd.ai** landing page. This breakdown maps exactly what the user sees and feels at every stage of the scroll, balancing the highly chaotic, organic energy of the 3D orb with a stark, dark-mode layout.

Phase 1: The Catalyst (0% Scroll — Page Load)

The Visual:

The page loads into a pure, pitch-black void. No header, no text, just an expansive dark canvas.

The Action:

- **The Swarm:** Glowing, mist-covered sub-particles drift lazily into the viewport. Suddenly, an invisible gravitational pull yanks them toward the exact center of the screen.
- **The Formation:** The particles snap together with a sharp, satisfying easing curve, forming the pulse-shaped **addictd.ai logo**.
- **The Storm:** The logo immediately begins to spin at high velocity, distorting into a fierce, localized tornado of light. Deep purple lightning arcs across the center.
- **The Birth of the Orb:** The tornado violently compresses and then blooms outward, revealing the final 3D Orb. It is heavy, volumetric, and alive—filled with swirling purple mist and erratic electrical zaps.

Phase 2: The Anchor & First Refraction (5% – 25% Scroll)

The Visual:

As the user makes their first scroll down, the sheer scale of the orb is established. A razor-sharp, minimalist header fades in at the top (and will now gracefully hide/show based on scroll direction).

The Action:

- **The Sliding Door:** The orb physically shifts slightly off-center (or the camera pushes in close), creating asymmetrical negative space on the screen.
- **The Refraction:** A thick, bright ray of refracted light pierces out from the core of the orb. It hits an invisible plane and forms a glowing "bubble" or node.
- **The Feature Reveal:** This node instantly expands into a sleek, high-contrast UI card displaying the **Retention Heat Map**. The UI card utilizes severe, clean architectural lines and a monochromatic palette to heavily contrast the swirling purple chaos of the orb behind it.

Phase 3: The Data Ecosystem (25% – 70% Scroll)

The Visual:

The user is now deep into the product features. The screen is a mix of the mesmerizing 3D object and premium, legible data visualizations.

The Action:

- **Orbital Rotation:** With every scroll tick, the orb rotates dynamically on a 3D axis. The internal lightning strikes react to the scroll velocity—scrolling faster makes the internal storm more aggressive.
- **Continuous Refraction:** The first UI card dissolves into mist as a new ray of light shoots out from a different angle of the rotating orb.
- **Sequential Feature Cards:** New data cards materialize out of these refractions:

    - *Stop 1:* **Predictive Insights** (showing drop-off zones).
    - *Stop 2:* **Revenue Charts** (showing creator monetization tracking).
- **The Swarm Effect:** Smaller, secondary nodes (like the orbiting moons in the storyboard) begin to circle the main orb. These act as passive visual elements housing quick-hit stats or social links, rotating behind and in front of the main sphere to emphasize depth.

Phase 4: The Climax & Shatter (70% – 90% Scroll)

The Visual:

The user approaches the bottom of the landing page. The data cards have all transitioned away. The orb is back in the absolute dead center of the screen, but it is highly unstable.

The Action:

- **Overload:** The orb begins to vibrate. The purple mist inside thickens, and the lightning arcs become a constant, bright glow.
- **The Fracture:** At a specific scroll trigger threshold, the surface tension of the orb breaks.
- **The Shatter:** The orb explodes entirely, shattering back into the thousands of individual sub-particles from the opening sequence. They rush toward the camera, breaking the 4th wall, before rapidly decelerating.

Phase 5: The Fog & The Hook (90% – 100% Scroll)

The Visual:

The chaotic energy completely dissipates, leaving a calm, highly structured final viewport.

The Action:

- **The Mist Transition:** The shattered particles diffuse outward, filling the background of the final section with a slow-moving, dark purple volumetric fog.
- **The Final CTA:** Out of this fog, the final Call to Action cleanly emerges in stark, bold typography. The footer locks into place at the very bottom of the viewport.
- **The Loop:** If the user scrolls back up, the fog aggressively pulls back together, re-forging the orb to reverse the journey.

The Clarity:

With the clarity of vision we have:
This sample completely shifts the paradigm. Charlie didn't just give you a static mockup; he gave you a fully interactive, math-heavy Canvas 2D prototype. This is massive because it completely clarifies the mechanics of the scroll experience.

The narrative has evolved from "refracted light" to a **"Containment Breach."** This is a much stronger, more visceral metaphor for a data-heavy application. As the user scrolls, the sheer volume of data and intelligence inside this orb is putting it under pressure, causing the glass to fracture.

Here is our updated, finalized storyboard and technical game plan, integrating the prototype's logic and Charlie's new "glass crack to smoke" directive:

The Updated Storyboard: The Containment Breach

1. The Catalyst (0% Scroll)

- **Action:** The particles swarm, form the addictd.ai pulse logo, spin into a chaotic tornado, and rapidly crystallize into the heavy, volumetric 3D orb.
- **Vibe:** Unchanged. High energy, establishing the core visual language.

2. The Pressure Builds (First Scroll)

- **Action:** As the user scrolls to the first feature, the orb physically rotates. Suddenly, a **super clean, sharp fracture** tears across the surface of the glass.
- **The Metaphor:** The orb can't contain the insights anymore.

3. The Smoke Forging (The Feature Reveal)

- **Action:** Deep purple, volumetric smoke violently vents out of the fresh crack. This smoke billows outward across the screen and rapidly condenses, forging the sleek, dark-mode **Retention Heat Map** UI card.
- **UI Connection:** The UI cards are no longer just floating; they are tethered to the orb by these trails of venting smoke (exactly like the `.mist-trail` in the prototype, but rendered with high-fidelity fluid dynamics).

4. The Chain Reaction (Continuous Scroll)

- **Action:** Scrolling further rotates the orb to a new angle. *Crack.* A second clean fracture appears. More smoke vents out, forging the **Timeline Analysis** card. This repeats. The orb becomes increasingly fractured, glowing brighter as more "pressure" is released through these cracks to form your feature cards.

5. The Climax & Shatter

- **Action:** Once all features are revealed, the structural integrity of the glass orb is gone. It completely explodes, shattering back into thousands of sub-particles.

6. The New Finale: Morph & Settle

- **Action:** The prototype introduces a brilliant ending. The shattered particles don't just disappear; they form a **Platform Selector**. The particles morph into the Discord logo, then YouTube, X, etc., controlled by user arrows.
- **The Pricing Drop:** Finally, the particles lose their energy and fall like sand to the bottom of the screen, creating a subtle glowing baseline that grounds the final Pricing Tier cards.

The Technical Reality Check (Senior Dev Perspective)

The provided HTML file is a masterpiece of native Canvas 2D math (using a Fibonacci sphere projection and 2D particle arrays). However, if Charlie wants **"super clean cracks in a glass orb"** with photorealistic smoke, Canvas 2D is going to hit a visual ceiling. It will look like a flat gradient rather than real glass.

To hit that premium brutalist, Awwwards-winning quality you are aiming for, we still need to build this in **React Three Fiber (R3F) within your Next.js environment.**

Here is how we translate the prototype's logic into the final stack:

- **The Glass Orb:** We need a custom Three.js `MeshPhysicalMaterial`. We will crank up the `transmission` (glass-like transparency), `ior` (index of refraction for bending light), and `thickness`.
- **The Clean Cracks:** Instead of generating 2D lines, we will use a custom shader to feed a "crack texture map" into the glass material's `normalMap` and `roughnessMap`. As the user scrolls, we animate the UV offset of this texture so the cracks appear to "grow" sharply across the 3D surface.
- **The Venting Smoke:** We can use WebGL particle systems positioned exactly at the coordinates of the cracks on the 3D sphere. We emit particles outward, and using `@react-three/drei`'s `<Html>` wrapper, we position your React UI cards exactly where the smoke trail ends.
- **The Logo Morphing:** R3F makes this incredibly smooth. We load the SVGs of the platform logos, extract their points, and use GSAP to animate the 3D particles from their chaotic shattered state into the structured logo formations.

This prototype is the perfect blueprint. It gives us the exact scroll timings, the camera angles, and the particle counts. Our job is simply to upgrade the rendering engine from Canvas 2D to a high-fidelity WebGL environment.

Would you like to start by setting up the Next.js/R3F boilerplate and focusing on getting that pristine `MeshPhysicalMaterial` glass look dialed in first?

The Stack

This is the make-or-break phase. For an Awwwards-caliber, high-performance site like this, the architecture needs to be flawless. If we bog down the main thread, that beautiful orb turns into a laggy slideshow.

Here is the definitive, production-ready tech stack, mapped exactly to the components of the addictd.ai landing page:

1. The 3D Engine & Visuals (The Core Experience)

- **React Three Fiber (R3F):** The primary wrapper. We will use this to declare our 3D scene, manage the camera, and handle the rendering loop within our React tree.
- **Three.js `MeshPhysicalMaterial`:** The glass orb. We will push the transmission, thickness, and clearcoat properties to the max to get that heavy, refractive crystal ball look that bends the light from the inside.
- **Custom GLSL (Fragment & Vertex Shaders):** The volatile energy. Standard materials can't do the tornado, the internal lightning, or the clean surface cracks. We will write custom shaders to handle the math for the chaotic volumetric smoke inside and the glowing fractures on the surface.
- **`@react-three/drei` (Specifically the `<Html>` component):** The UI bridge. This is crucial. We will use Drei's HTML helper to perfectly project the React-based feature cards (like the Retention Heat Map) out of the venting 3D smoke, pinning standard DOM elements to precise 3D coordinates.

2. The Choreography (Scroll & Animation)

- **GSAP (GreenSock):** The absolute gold standard for timeline sequencing. We will use it to handle the initial boot-up sequence (particles forming the logo into the orb).
- **GSAP `ScrollTrigger`:** The scroll engine. This replaces standard scroll listeners. We will pin the 3D canvas to the viewport and scrub a master GSAP timeline back and forth based on the user's scroll position. This single tool controls the "sliding door" effect, the orb's rotation angles, and the exact moments the glass cracks.
- **Framer Motion (Optional, for DOM UI):** While GSAP handles the heavy 3D lifting, Framer Motion is perfect for the standard UI layer—like the smooth staggering of the Pricing Tier cards at the end or the Platform Selector morphs.

3. The Foundation & Layout

- **Next.js (App Router):** The skeleton. It provides the perfect React foundation to house the canvas component while keeping the rest of the page (nav, footer, pricing) SEO-friendly and highly optimized.

- **Tailwind CSS:** The styling engine. To nail that "Premium Brutalist" aesthetic Charlie wants, Tailwind allows us to rapidly build those severe, high-contrast, dark-mode data cards and keep the CSS bundle microscopic.

4. The Infrastructure & Asset Delivery

- **Cloudflare (R2/CDN):** To get the orb looking photorealistic, we might need some heavy texture maps (like an HDRI for lighting or normal maps for the glass). Serving these assets externally ensures the initial page load is blazing fast. We just have to ensure the bucket's CORS policies are dialed in perfectly—specifically ensuring the `AllowedOrigins` array doesn't have any rogue trailing slashes blocking our local or production domains.
- **Vercel:** The deployment pipeline. Getting this live is as simple as making sure we actually commit the code before pushing to the main branch, letting Vercel handle the edge caching and Next.js optimization automatically.

This stack gives us absolute control over the physics, the rendering, and the DOM without sacrificing performance.

Before we start scaffolding the Next.js environment, do you want to handle the data visualizations inside those feature cards with raw CSS/SVG (like Charlie's prototype), or should we bring in a lightweight charting library to make those retention graphs pop?

Design Tokens:

Here is the complete **Design System & UI Tokens** section. I have formatted this specifically so you can drop it directly into your `addictd.ai.pdf` document.

This is written in a way that gives Antigravity exact mathematical values for the "Premium Brutalist" aesthetic, translating the legacy CSS into actionable Tailwind configuration logic.

Design System & UI Tokens (The "Premium Brutalist" Aesthetic)

To achieve the severe, high-contrast, and premium aesthetic, all UI elements layered over the 3D canvas must strictly adhere to the following design tokens. Do not deviate from these exact color hexes, opacities, and font families. These should be configured in your `tailwind.config.js`.

1. Typography Engine

We are using a dual-font system.

- **Display / Headings / Large Numbers:** `Syne`
  - *Weights:* 400, 500, 700, 800.

    - *Usage:* Only use for main Hero titles, pricing plan names, massive data numbers (e.g., Hook Score "87"), and the logo.
- **Body / UI Elements / Badges:** Outfit
    - *Weights:* 300, 400, 500, 600.
    - *Usage:* Use for all secondary text, paragraphs, navigation, "pill" tags, and small UI labels.

2. The Core Color Palette

The environment is a dark void, pierced by highly saturated, electric purples.

- **Backgrounds & Void:**
    - --bg-void: #06040F (Use for the absolute background canvas).
    - --bg-panel-dark: #0D0A1F (Use for standard, non-glass opaque panels like Pricing Cards).
    - --bg-panel-inset: #1A1428 (Use for nested boxes inside panels).
- **The Addictd Purples:**
    - --pur-deep: #3D1A78 (Shadows, deep gradients).
    - --pur-dark: #5B21B6 (Borders, inactive states).
    - --pur-base: #8B5CF6 (Primary brand color, standard buttons).
    - --pur-bright: #A78BFA (Highlights, glowing text, active states).
    - --pur-glow: #C084FC (Used specifically for SVG gradient stops and text-shadows).
- **Typography Colors:**
    - --text-main: #EEEAF8 (Primary body and headings).
    - --text-dim: #8A8499 (Secondary text, subtitles, inactive nav links).
- **Semantic UI (For Data Charts & Suggestions):**
    - *Positive/Success:* Base #10B981, Highlight #6EE7B7, BG rgba(16,185,129,.1)
    - *Warning/Medium:* Base #F59E0B, Highlight #FCD34D, BG rgba(245,158,11,.1)
    - *Critical/Drop-off:* Base #EF4444 (No background, used for harsh timeline drops)

3. Glassmorphism & Material Formulas

The feature cards forged from the smoke must look like physical, frosted glass floating in 3D space. Use this exact formula for the .feat-card containers:

- **Background Gradient:** linear-gradient(135deg, rgba(35,15,75,.88) 0%, rgba(60,30,130,.75) 50%, rgba(80,40,160,.7) 100%)
- **Backdrop Blur:** blur(28px)
- **Border:** 1px solid rgba(255,255,255,.12) (Top inner rim lighting) coupled with an outer glow border of rgba(139,92,246,.18).
- **Box Shadow (The Glow):** 0 0 70px rgba(167,139,250,.35), 0 0 130px rgba(139,92,246,.18), inset 0 0 30px rgba(200,170,255,.07)

4. UI Component Guidelines

- **Borders:** Most structural borders (like the header or pricing grid) should be razor-thin and low-opacity: `1px solid rgba(139,92,246,.15)`.
- **Border Radius:** * Feature Cards/Pricing Cards: `24px` / `rounded-3xl`
    - Inner Data Boxes/Buttons: `10px` to `14px`
    - Pills/Tags: `100px` / `rounded-full`
- **Buttons:** Primary CTAs should use a sharp linear-gradient `(135deg, #8B5CF6 0%, #A78BFA 100%)` with a `scale(1.03)` transform and a heavy drop-shadow `0 0 40px rgba(139,92,246,.6)` on hover.

Asset Requirements:

Here is the complete **Asset Requirements & Sourcing** section, formatted so you can drop it directly into your `addictd.ai.pdf` document right after the Design System.

Asset Requirements & Sourcing

Because this landing page relies heavily on WebGL and custom shaders, standard static images (PNGs/JPGs) are insufficient for the core interactive elements. The following assets must be explicitly provided or mathematically generated, as the `MeshPhysicalMaterial` and the particle morphing engine depend on them to function.

1. Platform Logos (Vector Data for Particle Morphing)

To achieve the Phase 4 "Shatter & Morph" sequence, the 3D particles need precise coordinates to snap into the shapes of social media platforms.

- **Required Format:** `SVG` (Scalable Vector Graphics).
- **Specifications:** These must be single-color, clean paths with no embedded raster images or complex clipping masks. The system will parse these SVG paths to extract raw coordinate data (point sampling) so the 3D particles know exactly where to travel.
- **Required Platforms:**
    - Discord
    - YouTube
    - X (Twitter)
    - Instagram
    - TikTok

2. Brand Assets

- **The Addictd Logo:** The initial pulse/waveform logo that forms during the boot-up sequence must also be provided as a clean `SVG`. The particle engine will use this as the anchor coordinates for Phase 1.

3. 3D Environment & Material Maps

To make the `MeshPhysicalMaterial` orb look like heavy, photorealistic fractured glass, we need specific texture maps.

- **HDRI (High Dynamic Range Image):** A subtle, dark-studio HDRI map (`.hdr` or `.exr` format) is required to provide realistic environmental lighting and reflections across the surface of the glass orb.
- **Fracture/Normal Maps (Optional but Recommended):** While cracks can be generated purely via GLSL math (procedural generation), providing a high-resolution "shattered glass" Normal Map and Roughness Map will significantly enhance the realism of the fractures when the orb breaks.

4. Asset Hosting & Delivery (Cloudflare R2)

All heavy assets (HDRIs, complex SVGs, texture maps) must be hosted externally to ensure the Next.js initial bundle size remains extremely small.

- **Target:** Cloudflare R2 (or an equivalent CDN).
- **CORS Requirement:** You must configure the CORS policy on the bucket to allow `GET` requests from your local development environment (`localhost`) and the production Vercel domain. Ensure the `AllowedOrigins` array is perfectly formatted (e.g., `https://addictd.ai` without a trailing slash).